PENSE-BÊTE · TECHNIQUE ET COMMANDES

Programmation et développement sous Debian

07-62 Référence Debian - Programmation et scripts shell — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. Pour garantir la portabilité des scripts, il est conseillé d'utiliser /usr/bin/sh et d'éviter les bashismes ou zshismes.
2. Les programmes compilés manuellement doivent être installés dans les répertoires /usr/local ou /opt pour éviter les collisions avec le système.
3. Le développement de paquets Debian doit respecter la Charte Debian en utilisant des outils comme debuild et sbuild dans un environnement isolé.

Synthèse de l'environnement

Notion	À retenir
Documentation et outils	`man` (paquets `manpages`, `manpages-dev`) ; `info` pour les outils **GNU** (collection d'outils comme `gcc`, `sed`). Archives `contrib`/`non-free` parfois nécessaires (documentations **GFDL**).
Installation et versions	Sources compilées dans `/usr/local` ou `/opt` (`--prefix=/usr/local`). Sans les options d'origine, `checkinstall` permet une désinstallation propre. Ne jamais nommer un exécutable `test`.
Langages interprétés	Tâches simples : `dash`. Tâches complexes de zéro : `python3`. Retouche de code existant : `perl`, `tcl` ou `ruby`. Partie critique lente : réécrire en **compilé**.

Synthèse de la programmation shell

Notion	À retenir
Compatibilité POSIX	Utiliser `/usr/bin/sh` (lien vers `/usr/bin/dash`) pour le non-interactif ; éviter **bashismes** ou **zshismes** pour la portabilité.
Paramètres et expansions	Paramètres : `$0` (nom), `$1` (1er paramètre), `$#` (nombre), `$?` (retour). Expansions : `${var:-string}`, `${var:=string}`, `${var:?string}`.
Opérateurs et boucles	Succès = `0` ; erreur ≠ `0`. `[` (ou `test`) évalue `-e`, `-d`, `-f`, `-z`, `-n`. Boucles : `for`, `while` (tant que vrai), `until` (tant que faux).
Traitement et utilitaires	Tokens via espaces/métacaractères. Substitution : `$(cmd)` ou `` `cmd` ``. Paquets essentiels : dash, coreutils, grep, sed, mawk, debianutils.

Synthèse du développement C et outils

Notion	À retenir
Langage C	Installer `gcc`, `build-essential`, `libc6-dev` (fichiers d'en-tête/routines). Utiliser `snprintf(3)` et `strncpy(3)` contre les débordements. Option `-lm` pour `libm`.
Analyseurs	Flex (lexical, compatible **Lex**) et Bison (syntaxique **LALR**, compatible **Yacc**). Flux : `main()` → `yyparse()` → `yylex()`.
Construction	`Make` (via `Makefile`). Autotools (**Autoconf**, **Automake**, **Libtool**) pour la portabilité. Alternatives rapides : `CMake`, `Ninja`, `Meson` (utilise `Ninja` comme dorsal).

Synthèse de l'analyse et du débogage

Notion	À retenir
Analyse statique, formatage	Lint : `shellcheck` (scripts), `pyflakes3`/`pylint` (Python), `rubocop` (Ruby) ; Formatage : `indent` (C), `black` (Python), `perltidy` (Perl) ; Index : `ctags`.
Débogueur GDB	Compilation avec `-g` ; Commandes : `b` (arrêt), `run` (lancer), `next`/`step` (progresser), `p` (afficher une variable) ; Symboles : paquets `*-dbgsym` dans `debian-debug`.
Fuites mémoire, traçage	Fuites : `Valgrind`, `electric-fence`, `malloc(3)`, `duma` (tampon), `Leaktracer` (C++) ; Plantage : `bt` ou `core dump` ; Dépendances : `ldd`.

Développement web et conversion

Notion	À retenir
Interfaces CGI	Le programme utilise la variable d'environnement `QUERY_STRING` pour les paramètres ; sa **sortie standard** est envoyée au navigateur.
Conversion de langages	AWK vers PERL via `a2p(1)` ; FORTRAN 77 vers C/C++ via `f2c(1)` ; NASM vers Assembleur GNU via `intel2gas(1)`.

Synthèse de la création de paquets

Notion	À retenir
Processus de création	Utiliser `debuild` et `sbuild` ; respecter la **Charte Debian** (`debian-policy`) ; utiliser un environnement isolé comme **chroot**.
Débogage et compilation	Installer les paquets `*-dbgsym` (archive `debian-debug`) ; sinon, reconstruire avec `apt-get source` après avoir installé `fakeroot`, `devscripts` et `build-essential`.

Pièges et confusions

- ⚠ Utiliser le nom test pour un fichier exécutable.
- ⚠ Confondre le langage C et le shell car le chiffre zéro signifie Vrai en shell alors qu'il signifie Faux en C.
- ⚠ Utiliser echo au lieu de printf pour les séquences d'échappement.
- ⚠ Utiliser des fonctions obsolètes comme sprintf ou strcpy au lieu de snprintf ou strncpy.
- ⚠ Ne pas utiliser checkbashisms pour vérifier la portabilité des scripts.

Sigles et définitions

POSIX standard qui définit les interpréteurs de commandes compatibles pour assurer la portabilité des scripts

bashisme syntaxe spécifique à l'interpréteur bash qu'il faut éviter pour garantir la portabilité

PID identifiant d'un processus utilisé pour désigner le script ou la tâche de fond

DFSG règles de la Debian Free Software Guidelines concernant la conformité de certaines documentations

REPL environnement de type READ + EVAL + PRINT + LOOP utilisé pour tester des langages

GTK bibliothèque permettant de créer des boîtes de dialogue graphiques via l'outil zenity

LALR type d'analyseur syntaxique compatible avec Yacc utilisé par des générateurs comme Bison

CGI interface permettant à des programmes sur un serveur web de générer des pages dynamiques interactives

Auto-test

1. Quel paquet permet de désinstaller proprement un logiciel compilé sans les options de configuration d'origine ?
2. Quelle variable d'environnement est utilisée par une interface CGI pour les paramètres ?
3. Quels sont les deux outils de génération syntaxique mentionnés ?
4. Quelle commande permet de voir les dépendances d'un programme ?
5. Comment appelle-t-on la collection d'outils comme gcc ou sed ?

Réponses : **1.** checkinstall · **2.** QUERY_STRING · **3.** Flex et Bison · **4.** ldd · **5.** GNU

Complément+ : connaissance standard ajoutée, absente de la fiche. À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.